Perceptron

Number of input nodes = number of features
You can understand it similar to linear equation

Importance of weights

Say you keep your hand on hot plate
→ our body receives msg that it has high temp → Temperature is feature
There would be many other features like weather, moisture but our body has high weight for hot plate temp.
Say you get msg from brain to remove hand
weights are slope of curve

Importance of bias

→ It is intercpt and hence does not force model to pass trough origin
→ Say weight are randomly intialized to zero it does no deactivates the neurons.

Training perceptron

Step1: Start with random weights & bais

x_1 : Temperature $\quad w_1 = 0$
x_2 : weight $\quad w_2 = 0$
$b = 0$

Step2 feed training example

Temperature	weight	Actual output
8	2	1 (Drop object)

$$z = w_1 x_1 + w_2 x_2 + b$$
$$z = 0$$

Step 3: compute error

Actual = 1 . prediction = 0 error = 1-0 = 01

Step4: Update weights

$w_{new} = w_{old} + \eta \times Error \times x$

η = learning rate (say 0.1)

$w_1 = 0 + 0.1 \times 1 \times 8 = 0.8$

$w_2 = 0 + 0.1 \times 1 \times 2 = 0.2$

$b = 0 + 0.1 \times 1 = b + \eta \times Error = 0.1$

Perceptron understand temp contributed much more

Step5 Next example

Temperature	Weight	Actual
2	9	0

$z = 0.8(2) + 0.2(9) + 0.1$

$= 3.5$

using step function

prediction = 1

Actual = 0

Error = -1

update weight again

$w_1 = 0.8 + 0.1(-1)\ 2$

$= 0.6$

$w_2 = 0.2 + 0.1(-1)\ 9$

$= -0.7$

$b = 0.1 - 0.1 = 0$

Object's weight importance was reduced bcoz u can hold little heavy object too

Step6 : Repeat

What is use of activation function

An activation function decides if neuron should "fire" & more importantly, introduces non linearity so neural network can learn more complex patterns

Say we dont use activation function

$$y_1 = w_1 x_1 + b_1$$

$$y_2 = w_2(y_2) + b_2 \quad = w_2(w_1 x_1 + b_1) + b_2$$

$$\Rightarrow w_1 w_2 x + (w_2 b_1 + b_2)$$

It is just linear equation.

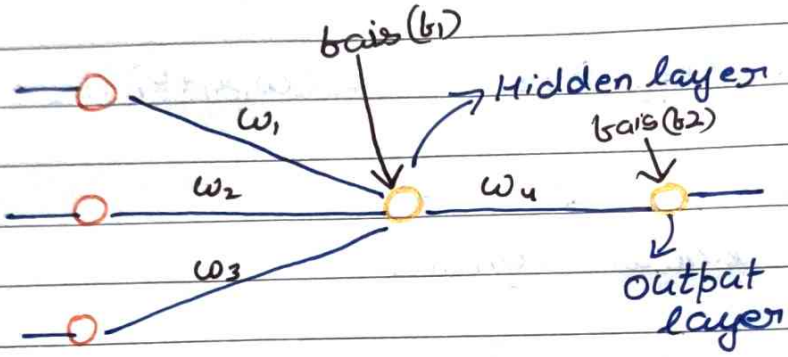
Input → linear → Activation → Linear → activation → output

Bais is at hidden layer node & output node

Multi-layered Perceptron model
[Ann]

1. Forward propagation
2. Backward propagation → Geoffrey Hinton
3. Loss function
4. optimizers
5. Activation function

Multi layered NN



IQ	Studyhour	Play hour	O/P
95	4	4	1
100	5	2	1
95	2	7	0

2 layered NN

Forward propogation

$$z = x_1\omega_1 + x_2\omega_2 + x_3\omega_3 + b_1$$

$$z = \sum_{i=1}^{n} \omega_i^T x_i + b$$

Activation (z)

This calculation happens in every neuron

$[\omega_1, \omega_2, \omega_3] = [0.01, 0.02, 0.03)$ $\quad b_1 = 0.001$

first record

Step1 $95 \times 0.01 + 4 \times 0.02 + 4 \times 0.03 + 0.001$

$= 1.151$

Step2 = Activation (say sigmoid)

O_1 = Activation(z) = $\frac{1}{1+e^{-z}} = \frac{1}{1+e^{-1.191}} = 0.769$

Activation fuction → activates newron
& deactivates newron

Hidden layer 2:

Say $w_4 = 0.02$
$b_2 = 0.03$

$z = O_1 \times w_4 + 0.03$

$0.759 \times 0.02 + 0.03$

$= 0.04518$

Activation(z) = $\frac{1}{1+e^{-0.04518}} = 0.51129$

$O_2 = 0.51129$

Loss function

Predicted output = 0.51129

Loss = (1 − 0.51129) = error

$\approx 0.49 \Rightarrow$ error → we need to reduce error

To minimize ~~activate~~ weights updation we do weight updation via backward proporgation

Loss function vs cost function

$(y-\hat{y})^2$ → for every point

$\sum_{i=1}^{n} (y-\hat{y})^2$